**Project Design Phase-II**
**Technology Stack (Architecture & Stack)**

| | |
|---|---|
| Date | 1 March 2026 |
| Team ID | GACSE-265 |
| Project Name | AI Financial Advisor |
| Maximum Marks | 4 Marks |

**Technical Architecture:**

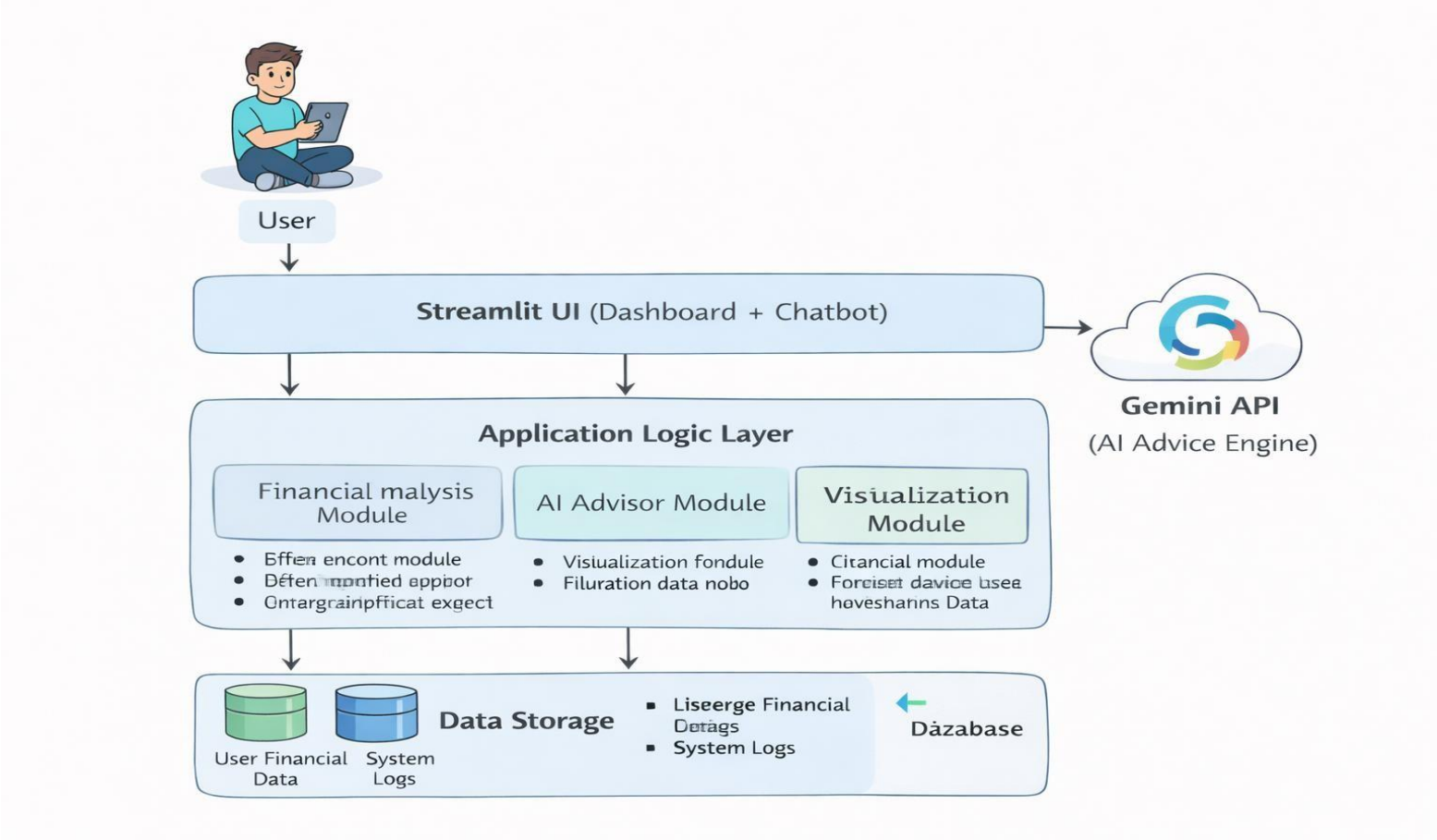

**Table-1 : Components & Technologies:**

| S.No | Component | Description | Technology |
|---|---|---|---|
| 1. | User Interface | Interface for users to input financial data and view insights through dashboard and chatbot | Streamlit Web UI |
| 2. | Application Logic-1 | Financial computation module to calculate savings ratio, debt ratio, and emergency fund | Python |
| 3. | Application Logic-2 | AI advisory module generating financial guidance and insights | Python + Gemini API |
| 4. | Application Logic-3 | Visualization module generating charts and financial reports | Python, Matplotlib, Seaborn |
| 5. | Database | Storage of user financial records and profile information | SQLite / Local Data Storage |

| | | | |
|---|---|---|---|
| 6. | Cloud Database | Optional cloud-based storage for scalability | Google Cloud Storage / Firebase |
| 7. | File Storage | Storage of configuration files, logs, and datasets | Local File System |
| 8. | External API-1 | AI language model used for generating financial advice | Google Gemini API |
| 9. | External API-2 | Optional financial market or economic data APIs | Financial Market APIs |
| 10. | Machine Learning Model | AI model generating personalized financial recommendations | Gemini Large Language Model |
| 11. | Infrastructure (Server / Cloud) | Deployment of application locally or on cloud platform | Local System / Google Cloud Platform |

**Table-2: Application Characteristics:**

| S.No | Characteristics | Description | Technology |
|---|---|---|---|
| 1. | Open-Source Frameworks | Frameworks used to build UI, analytics, and AI integration | Streamlit, Pandas, NumPy, Matplotlib |
| 2. | Security Implementations | Secure storage of API keys and protection of user data | .env configuration, API key protection |

| | | | |
|---|---|---|---|
| 3. | Scalable Architecture | Modular architecture separating UI, logic, AI services, and visualization | Python Modular Architecture |
| 4. | Availability | System accessible through web interface anytime | Streamlit Web Application |
| 5. | Performance | Efficient data processing and fast AI response generation | Optimized Python processing |